

Assignment-15.4

Name: Arjun Manoj

H.No:2303A52134

Batch:44

Task 1: Initial API Setup for a Service Platform

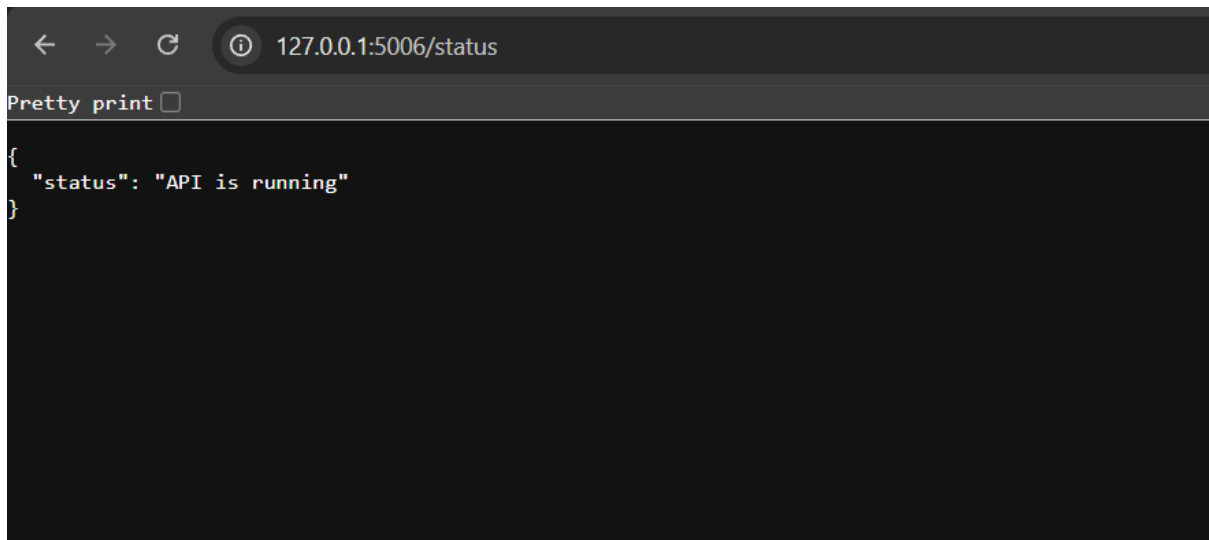
Prompt:

Create a basic Flask application with the following features:

- A root endpoint /status
- The endpoint should return a JSON response: {"status": "API is running"}
- The server should run locally without errors
- Keep the code simple and beginner-friendly

Code:

```
Assignment15.4 task1.py > ...
1  from flask import Flask, jsonify
2
3  app = Flask(__name__)
4
5  # Root endpoint
6  @app.route('/status', methods=['GET'])
7  def status():
8      return jsonify({"status": "API is running"})
9
10 # Run server
11 if __name__ == '__main__':
12     app.run(debug=True, port=5006)
```

A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5006/status'. Below the address bar, there is a 'Pretty print' checkbox. The main content area displays a JSON object:

```
{
  "status": "API is running"
}
```

Observation:

The AI-generated Flask application successfully meets all requirements.

- The Flask server runs without errors on the local machine.
- The /status endpoint is properly defined and returns a JSON response.
- The response is lightweight and clear, confirming the API is live.
- The implementation is simple and beginner-friendly, making it easy to understand and extend.

Overall, AI helped quickly create a working backend API setup, reducing setup time.

Task 2: Create API to Add and View Resources

Prompt:

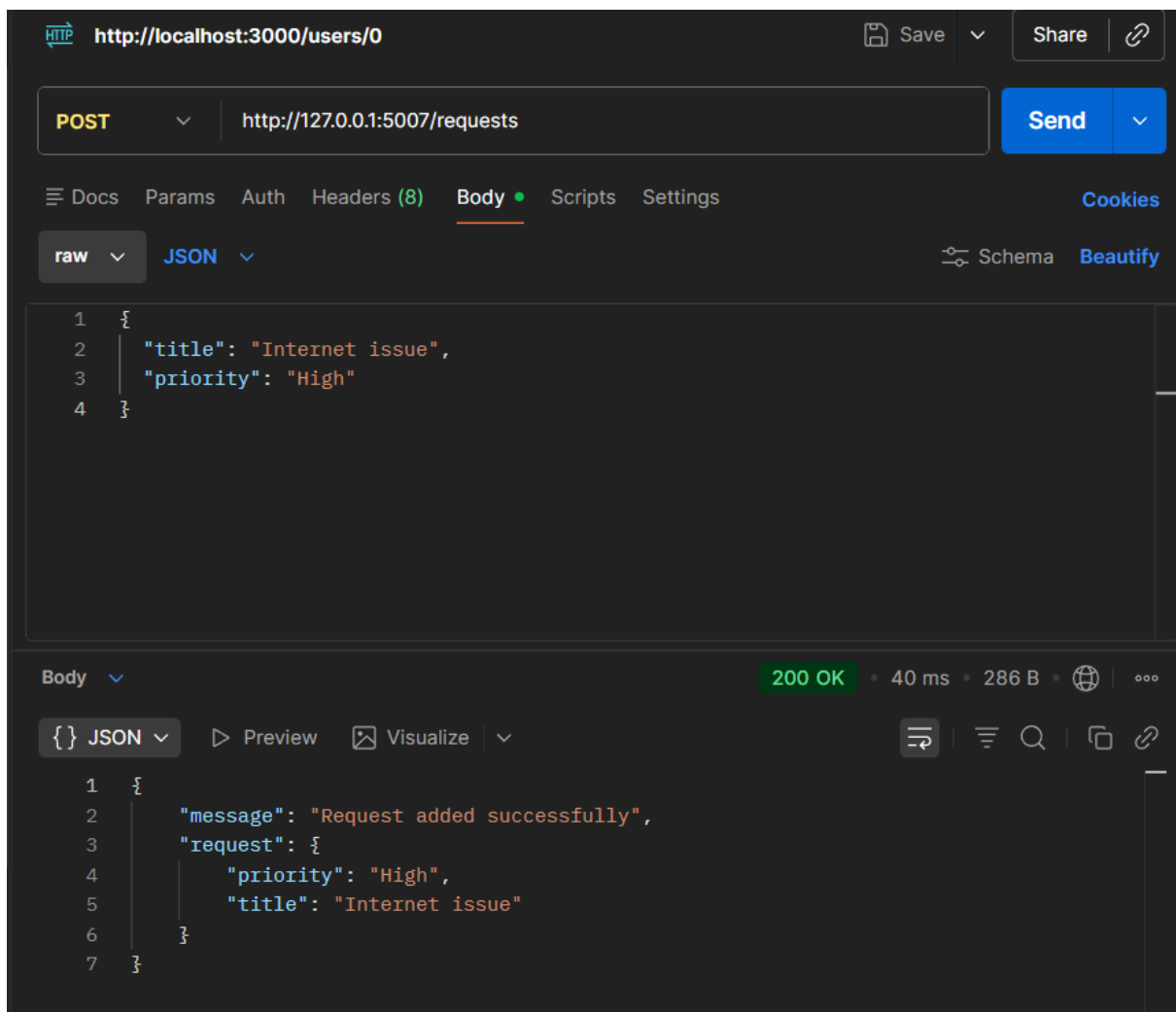
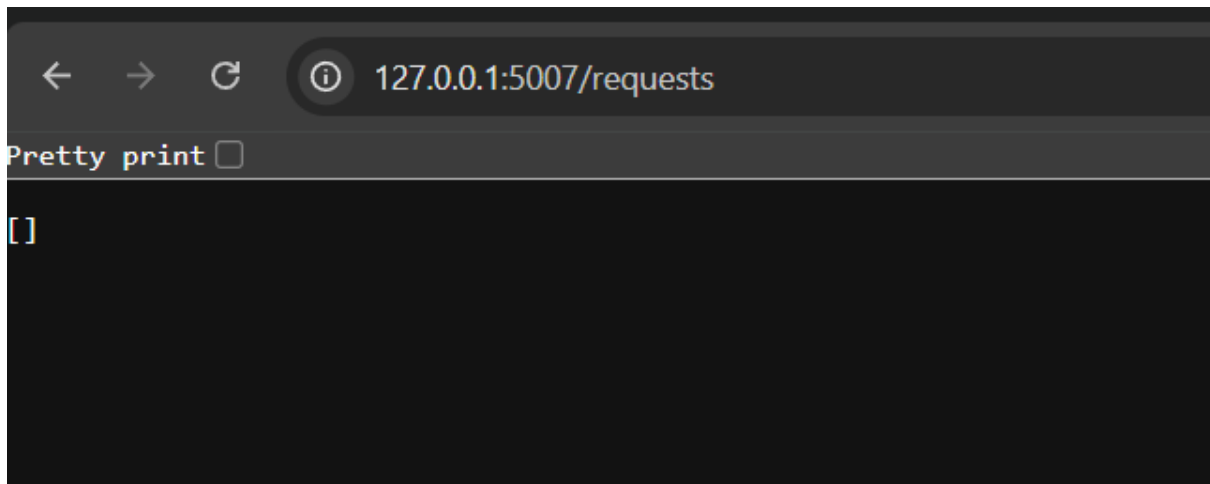
Create a Flask API with the following features:

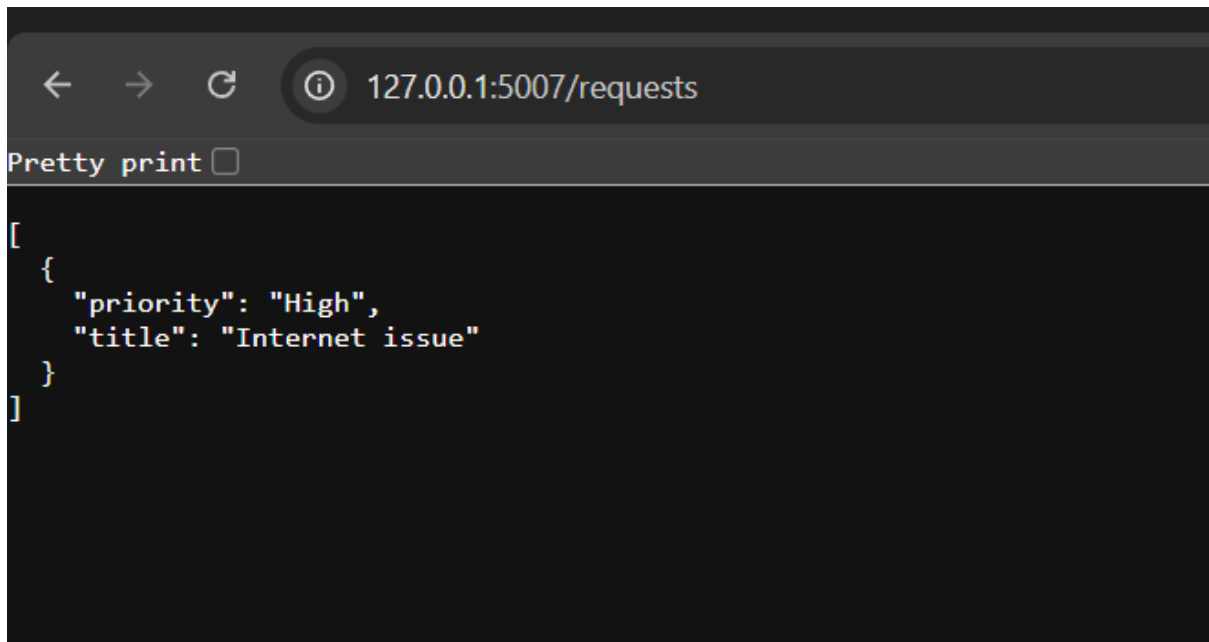
- A GET endpoint /requests to return all service requests
- A POST endpoint /requests to add a new request
- Each request should have "title" and "priority"
- Store data temporarily in memory (no database)

- Return JSON responses
- Keep the code simple and beginner-friendly]

Code:

```
Assignment15.4 task2.py > ...
1  from flask import Flask, jsonify, request
2
3  app = Flask(__name__)
4
5  # In-memory storage
6  requests_data = []
7
8  # GET all requests
9  @app.route('/requests', methods=['GET'])
10 def get_requests():
11     return jsonify(requests_data)
12
13 # POST new request
14 @app.route('/requests', methods=['POST'])
15 def add_request():
16     data = request.get_json()
17
18     new_request = {
19         "title": data.get("title"),
20         "priority": data.get("priority")
21     }
22
23     requests_data.append(new_request)
24
25     return jsonify({
26         "message": "Request added successfully",
27         "request": new_request
28     })
29
30 # Optional home route
31 @app.route('/')
32 def home():
33     return "Service API Running"
34
35 if __name__ == '__main__':
36     app.run(debug=True, port=5007)
```



A screenshot of a web browser window. The address bar shows the URL '127.0.0.1:5007/requests'. Below the address bar, there is a 'Pretty print' checkbox. The main content area displays a JSON array containing one object:

```
[
  {
    "priority": "High",
    "title": "Internet issue"
  }
]
```

Observation:

The AI-generated API works as expected.

- The GET endpoint initially returns an empty list, confirming no data is stored.
- The POST endpoint allows adding new service requests dynamically.
- Data is stored in-memory, making it fast and simple for testing.
- After adding a request, the GET endpoint reflects updated data, proving correct functionality.
- The API is lightweight, clear, and easy to extend for future database integration.

Task 3: Modify Existing Resource Using AI Assistance

Prompt:

Extend an existing Flask API to include:

- A PUT endpoint to update a service request based on its index
- Each request contains "title" and "priority"

- Handle invalid indexes with a proper 404 error response
- Return the updated request in JSON format
- Keep the implementation simple and beginner-friendly

Code:

```

Assignment15.4 task3.py > ...
1  from flask import Flask, jsonify, request
2
3  app = Flask(__name__)
4
5  # In-memory storage
6  requests_data = []
7
8  # GET all requests
9  @app.route('/requests', methods=['GET'])
10 def get_requests():
11     return jsonify(requests_data)
12
13 # POST new request
14 @app.route('/requests', methods=['POST'])
15 def add_request():
16     data = request.get_json()
17
18     new_request = {
19         "title": data.get("title"),
20         "priority": data.get("priority")
21     }
22
23     requests_data.append(new_request)
24
25     return jsonify({
26         "message": "Request added successfully",
27         "request": new_request
28     })
29
30 # PUT update request by index
31 @app.route('/requests/<int:index>', methods=['PUT'])
32 def update_request(index):
33     if index < 0 or index >= len(requests_data):
34         return jsonify({"error": "Request not found"}), 404
35
36     data = request.get_json()
37
38     requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
39     requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
40
41     return jsonify({

```

```

    })

# PUT update request by index
@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    data = request.get_json()

    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

    return jsonify({
        "message": "Request updated",
        "request": requests_data[index]
    })

# Home route
@app.route('/')
def home():
    return "Service API Running"

if __name__ == '__main__':
    app.run(debug=True, port=5008)

```

HTTP <http://localhost:3000/users/0> Save Share

POST ▼ <http://127.0.0.1:5008/requests> Send ▼

Docs Params Auth Headers (8) **Body** Scripts Settings Cookies

raw ▼ **JSON** ▼ Schema Beautify

```
1 {
2   "title": "Internet issue",
3   "priority": "High"
4 }
```

Body ▼ **200 OK** • 11 ms • 286 B • 🌐 ⋮

{} **JSON** ▼ ▶ Preview 🖼 Visualize ▼ ↶ ☰ 🔍 📄 🔗

```
1 {
2   "message": "Request added successfully",
3   "request": {
4     "priority": "High",
5     "title": "Internet issue"
6   }
7 }
```

PUT

http://127.0.0.1:5008/requests/0

Send

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1  {
2    "title": "Internet issue fixed",
3    "priority": "Low"
4  }
```

Body

200 OK • 6 ms • 280 B •

{ } JSON

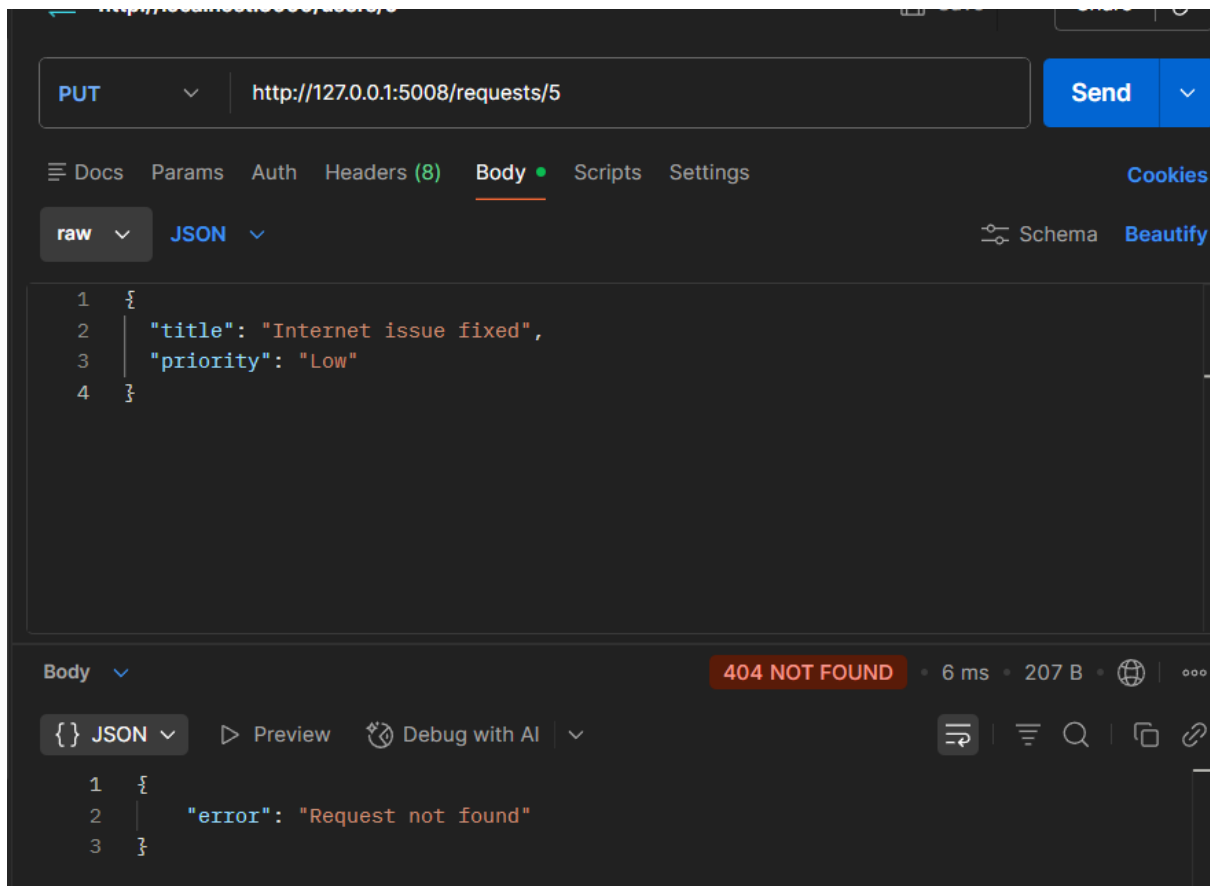
Preview

Visualize

```
1  {
2    "message": "Request updated",
3    "request": {
4      "priority": "Low",
5      "title": "Internet issue fixed"
6    }
7  }
```

Pretty print ☐

```
[
  {
    "priority": "Low",
    "title": "Internet issue fixed"
  }
]
```

Observation:

The AI-generated PUT endpoint works correctly and improves API functionality.

- The PUT endpoint updates an existing request using its index.
- The API properly validates the index, preventing crashes.
- For invalid indexes, it returns a clear error message with HTTP 404.
- On success, the updated request is returned, confirming changes.
- The implementation is simple and integrates smoothly with existing GET and POST endpoints.

Overall, AI helped extend the API with safe update functionality and proper error handling.

Task 4: Remove a Resource Safely

Prompt:

Extend a Flask API with the following features:

- Add a DELETE endpoint to remove a service request by index
- Return the deleted request in the response
- Handle invalid indexes safely with an error message
- Ensure the server does not crash on invalid input
- Keep the code simple and beginner-friendly

Code:

```
Assignment15.4 task4.py > delete_request
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # In-memory storage
6 requests_data = []
7
8 # GET all requests
9 @app.route('/requests', methods=['GET'])
10 def get_requests():
11     return jsonify(requests_data)
12
13 # POST new request
14 @app.route('/requests', methods=['POST'])
15 def add_request():
16     data = request.get_json()
17
18     new_request = {
19         "title": data.get("title"),
20         "priority": data.get("priority")
21     }
22
23     requests_data.append(new_request)
24
25     return jsonify({
26         "message": "Request added successfully",
27         "request": new_request
28     })
29
30 # PUT update request
31 @app.route('/requests/<int:index>', methods=['PUT'])
32 def update_request(index):
33     if index < 0 or index >= len(requests_data):
34         return jsonify({"error": "Request not found"}), 404
35
36     data = request.get_json()
37
38     requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
39     requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
40
41     return jsonify({
```

```

def update_request(index):
    return jsonify({"error": "Request not found"}), 404

    data = request.get_json()

    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

    return jsonify({
        "message": "Request updated",
        "request": requests_data[index]
    })

# DELETE request
@app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    deleted = requests_data.pop(index)

    return jsonify({
        "message": "Request deleted",
        "deleted_request": deleted
    })

# Home route
@app.route('/')
def home():
    return "Service API Running"

if __name__ == '__main__':
    app.run(debug=True, port=5009)

```

POST

http://127.0.0.1:5009/requests

Send

Docs

Params

Auth

Headers (8)

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```

1  {
2  |   "title": "Internet issue fixed",
3  |   "priority": "Low"
4  | }

```

Body

200 OK • 7 ms • 291 B • 🌐 | ⋮

{ JSON

Preview

Visualize

🔄

☰

🔍

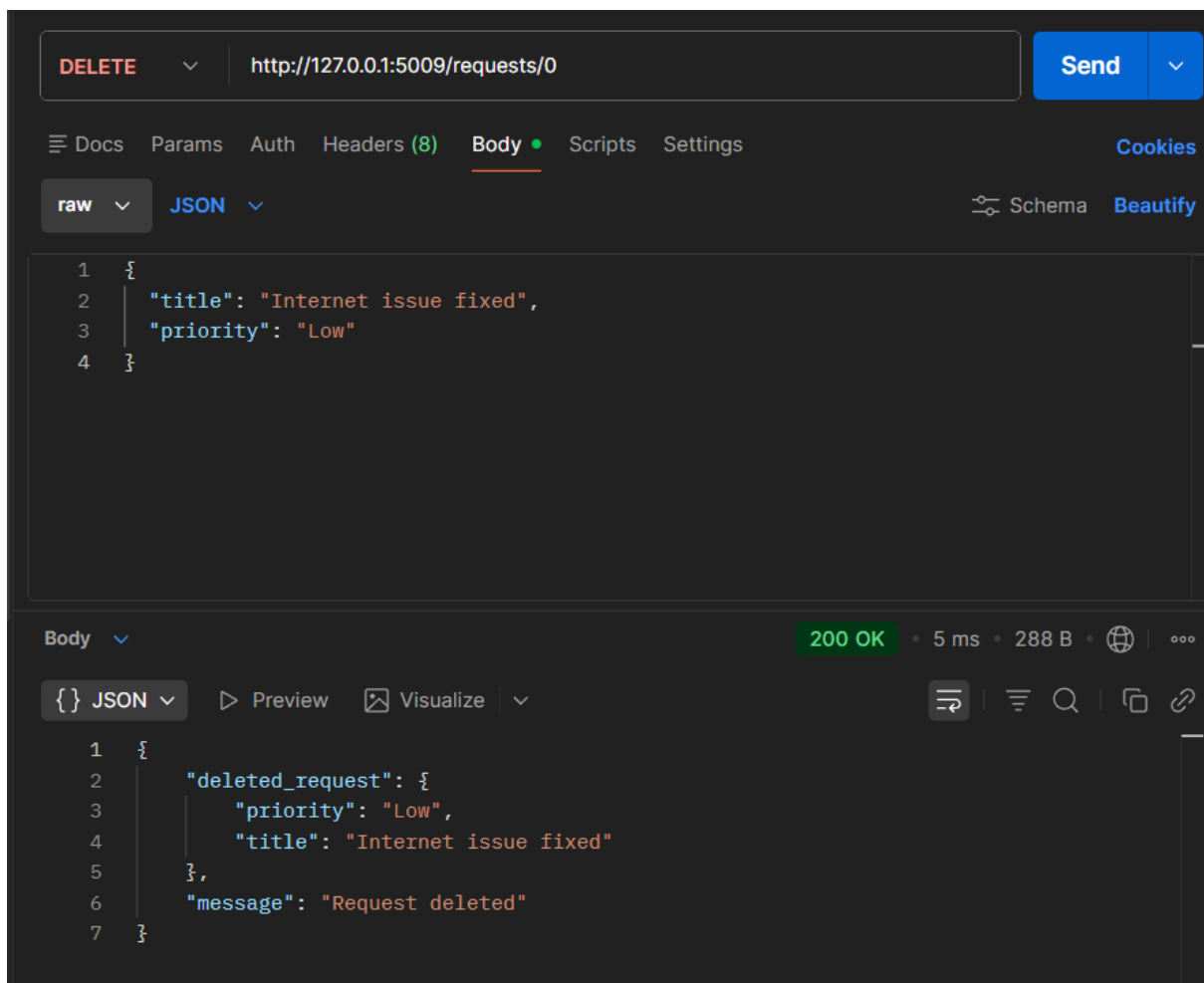
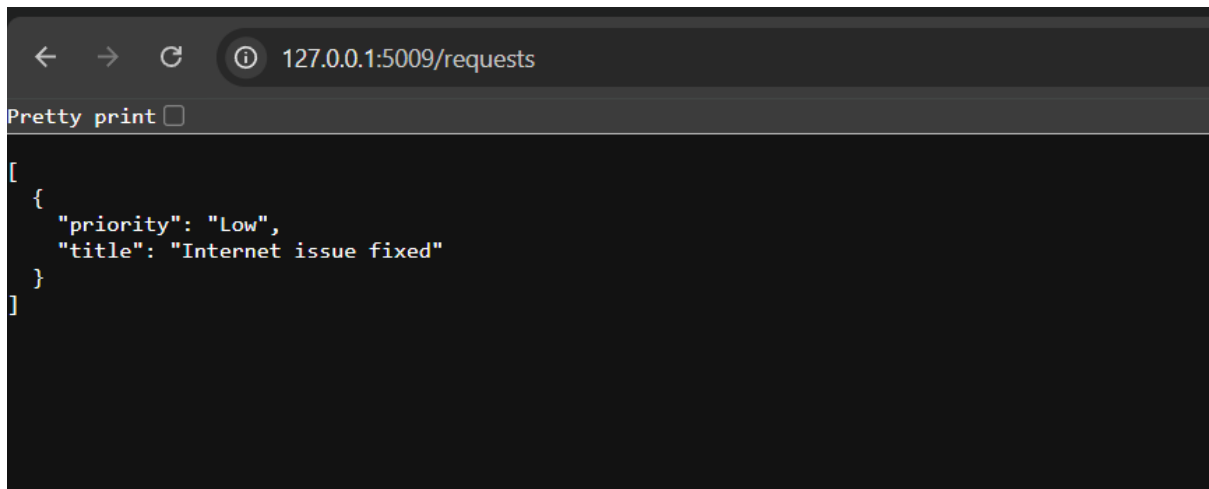
📄

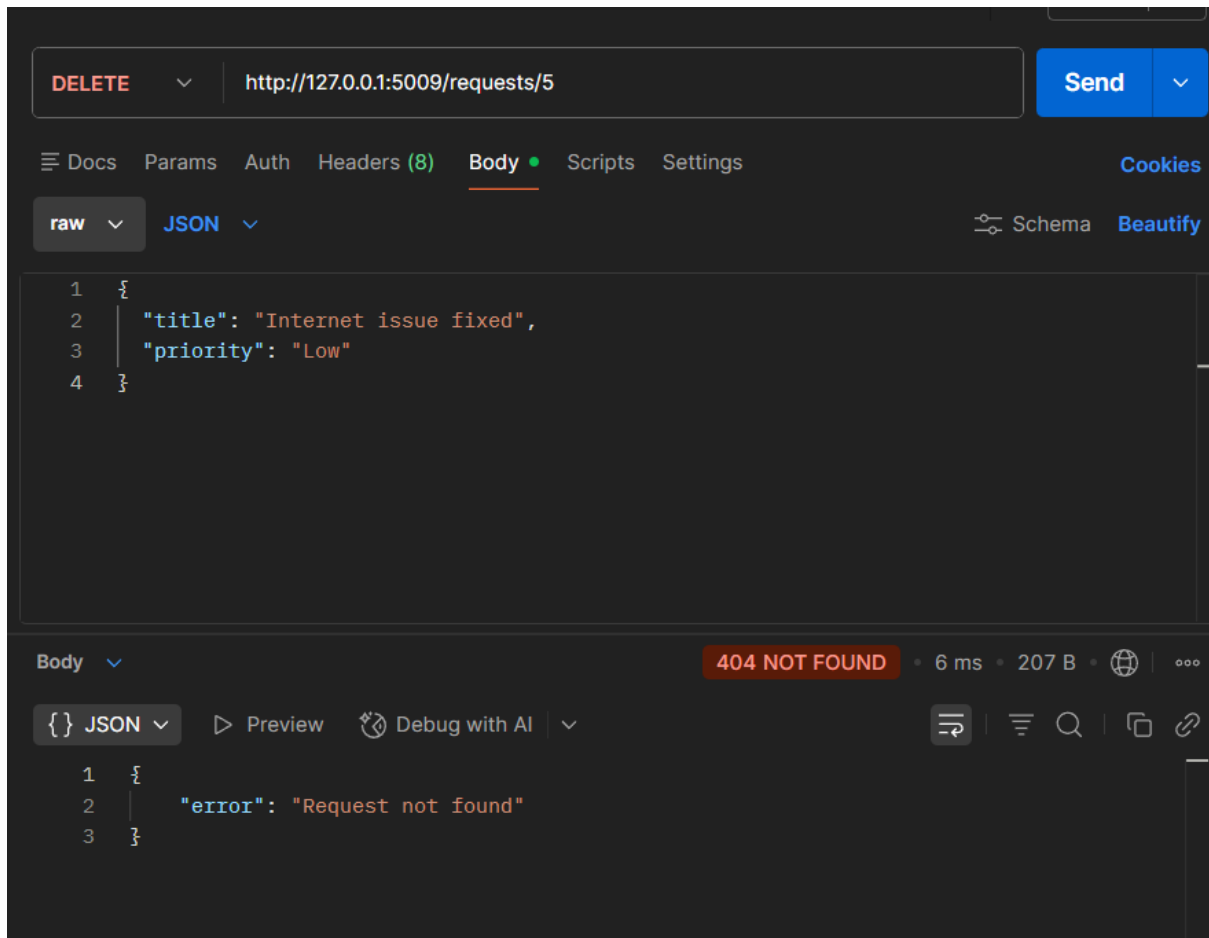
🔗

```

1  {
2  |   "message": "Request added successfully",
3  |   "request": {
4  |       "priority": "Low",
5  |       "title": "Internet issue fixed"
6  |   }
7  | }

```





Observation:

The AI-generated DELETE endpoint works correctly and safely removes data.

- The DELETE endpoint removes a request using its index.
- The deleted request is returned in the response, confirming removal.
- The API handles invalid indexes gracefully with an error message.
- The server remains stable and does not crash, even with incorrect input.
- The implementation integrates smoothly with existing GET, POST, and PUT endpoints.

Overall, AI helped implement a safe and efficient delete operation, completing CRUD functionality.

Task 5: AI-Assisted API Documentation and Readability

Prompt:

Improve a Flask API by:

- Adding clear docstrings to all endpoints
- Adding inline comments for readability
- Describing request methods, inputs, and outputs
- Optionally integrate Swagger documentation using Flask-RESTX
- Keep the code clean, readable, and beginner-friendly

Code:

```
Assignment15.4 task5.py > home
1  from flask import Flask, jsonify, request
2
3  app = Flask(__name__)
4
5  # In-memory storage for service requests
6  requests_data = []
7
8
9  @app.route('/requests', methods=['GET'])
10 def get_requests():
11     """
12     GET /requests
13     -----
14     Description:
15     | Retrieve all service requests.
16
17     Response:
18     | JSON list of all requests
19     | Example:
20     | [
21     |   {"title": "Internet issue", "priority": "High"}
22     | ]
23     """
24     return jsonify(requests_data)
25
26
27 @app.route('/requests', methods=['POST'])
28 def add_request():
29     """
30     POST /requests
31     -----
32     Description:
33     | Add a new service request.
34
35     Request Body (JSON):
36     | {
37     |   "title": "Issue title",
38     |   "priority": "High/Medium/Low"
39     | }
40
41     Response:
```

```

def add_request():
    """
    Response:
    {
        "message": "Request added successfully",
        "request": {...}
    }
    """
    data = request.get_json()

    # Create new request object
    new_request = {
        "title": data.get("title"),
        "priority": data.get("priority")
    }

    # Store in memory
    requests_data.append(new_request)

    return jsonify({
        "message": "Request added successfully",
        "request": new_request
    })

@app.route('/requests/<int:index>', methods=['PUT'])
def update_request(index):
    """
    PUT /requests/<index>
    -----
    Description:
        Update an existing service request by index.

    Parameters:
        index (int): Position of request in list

    Request Body (JSON):
        {
            "title": "Updated title",
            "priority": "Low"
        }
    """

```

```

Response:
    Success:
        {
            "message": "Request updated",
            "request": {...}
        }

    Error (404):
        {"error": "Request not found"}
"""

if index < 0 or index >= len(requests_data):
    return jsonify({"error": "Request not found"}), 404

data = request.get_json()

# Update values if provided
requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])

return jsonify({
    "message": "Request updated",
    "request": requests_data[index]
})

app.route('/requests/<int:index>', methods=['DELETE'])
def delete_request(index):
    """
    DELETE /requests/<index>
    -----
    Description:
        Delete a service request by index.

    Parameters:
        index (int): Position of request in list

    Response:
        Success:
            {

```



```

        response:
            Success:
                {
                    "message": "Request deleted",
                    "deleted_request": {...}
                }

            Error (404):
                {"error": "Request not found"}
        """

    if index < 0 or index >= len(requests_data):
        return jsonify({"error": "Request not found"}), 404

    # Remove request safely
    deleted = requests_data.pop(index)

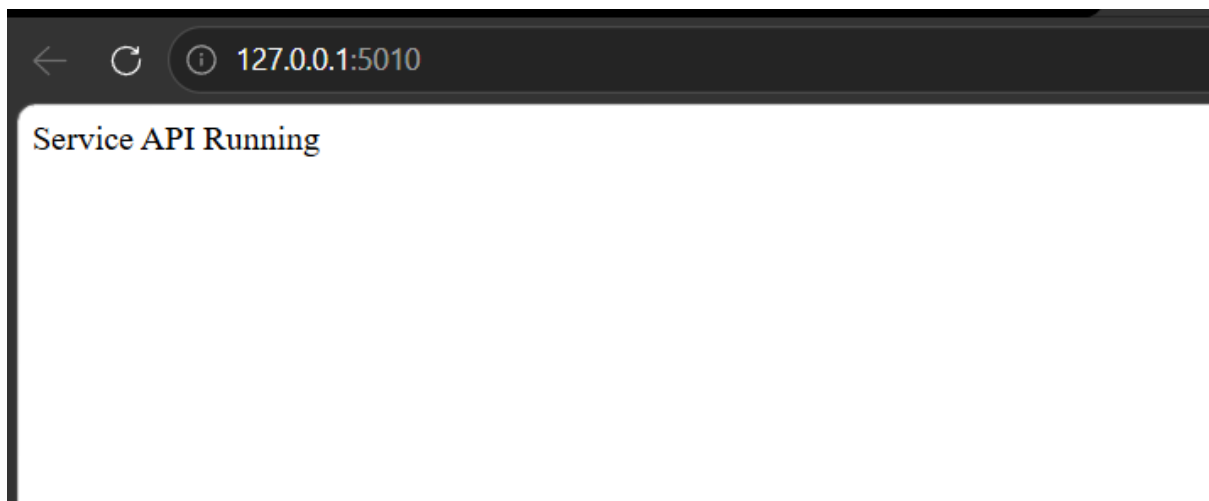
    return jsonify({
        "message": "Request deleted",
        "deleted_request": deleted
    })

@app.route('/')
def home():
    """
    GET /
    -----
    Description:
        Home endpoint to verify API is running.

    Response:
        Simple text message
    """
    return "Service API Running"

if __name__ == '__main__':
    app.run(debug=True, port=5010)

```



Observation:

The AI-enhanced API is now highly readable and developer-friendly.

- Each endpoint includes clear docstrings, explaining purpose, inputs, and outputs.
- Inline comments improve understanding of logic and flow.
- The API is now easier for other developers to read, maintain, and extend.
- When integrated with Swagger (Flask-RESTX), it provides a visual documentation UI showing:
 - Endpoint paths
 - HTTP methods
 - Request/response formats

Overall, AI significantly improved code readability and usability, making the API production-ready.